When usage exceeds capacity

Each GCell edge has capacity two on tiny_gr

The idea

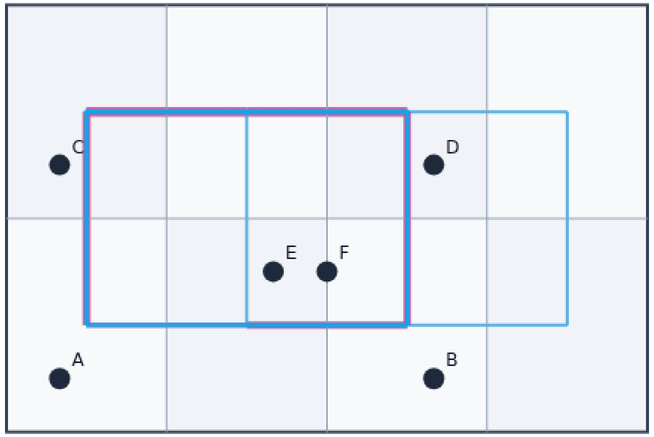
- For each edge e, overflow e equals max of zero and usage e minus capacity
- Total is the sum across edges
- Max is the worst edge
- Count is how many edges have positive overflow
- Sequential L-HV on all six nets should yield positive total overflow at cap two

Overflow

EDGE OVERFLOW

Overflow

Step 1 / 5 · def · module03-01-edge-overflow



Explanation

ov(e)=max(0, usage(e)−Cap) per edge.

- Not tile RUDY
- Router metric

```
Edge-local
```

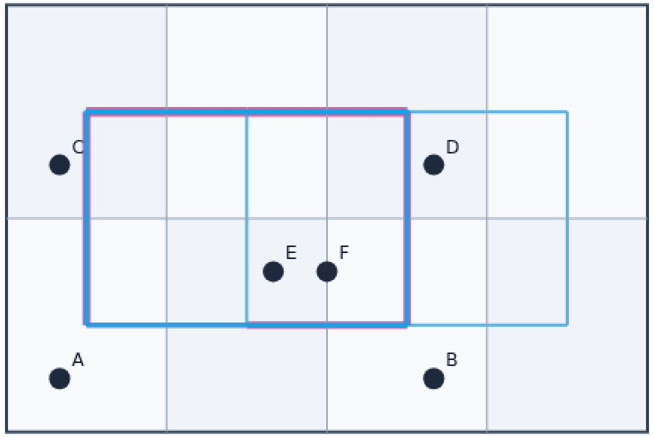
Overflow

Total overflow

EDGE OVERFLOW

Total overflow

Step 2 / 5 · total · module03-01-edge-overflow



Explanation

Sum over edges—primary regression scalar.

- Sum ov(e)
- Report every pass

```
spread L-HV≈2
```

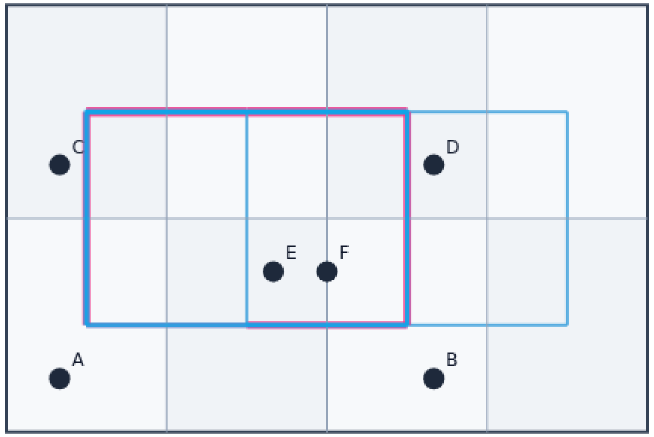
Total overflow

Max overflow

EDGE OVERFLOW

Max overflow

Step 3 / 5 · max · module03-01-edge-overflow



Explanation

Worst edge—catches hotspot corridors.

- Hot corridor
- max metric

```
max≈1
```

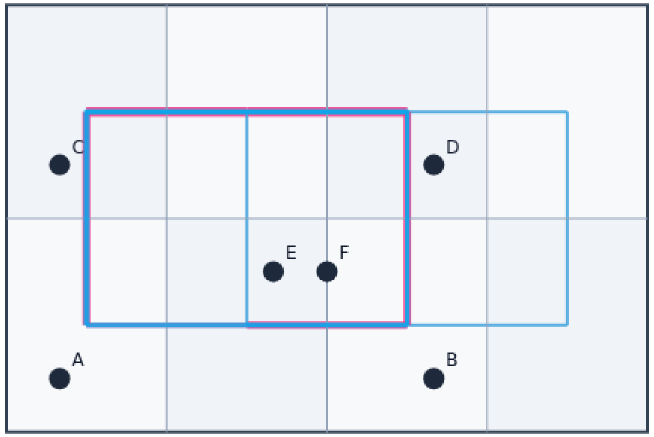
Max overflow

Overflow count

EDGE OVERFLOW

Overflow count

Step 4 / 5 · count · module03-01-edge-overflow



Explanation

How many edges exceed Cap.

- Triple report
- Same idea as congestion labs

```
count edges
```

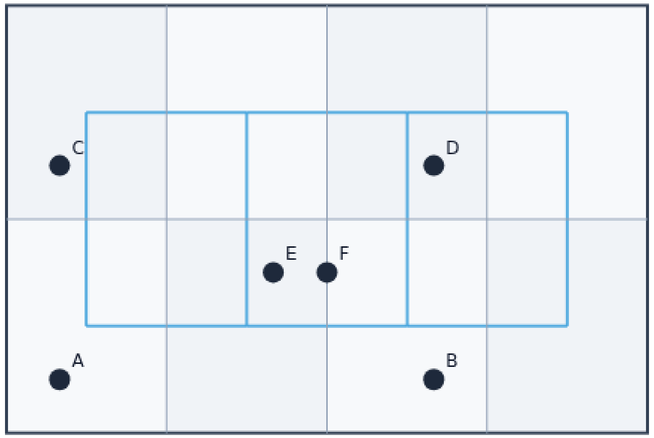
Overflow count

Hit targets

EDGE OVERFLOW

Hit targets

Step 5 / 5 · target · module03-01-edge-overflow



Explanation

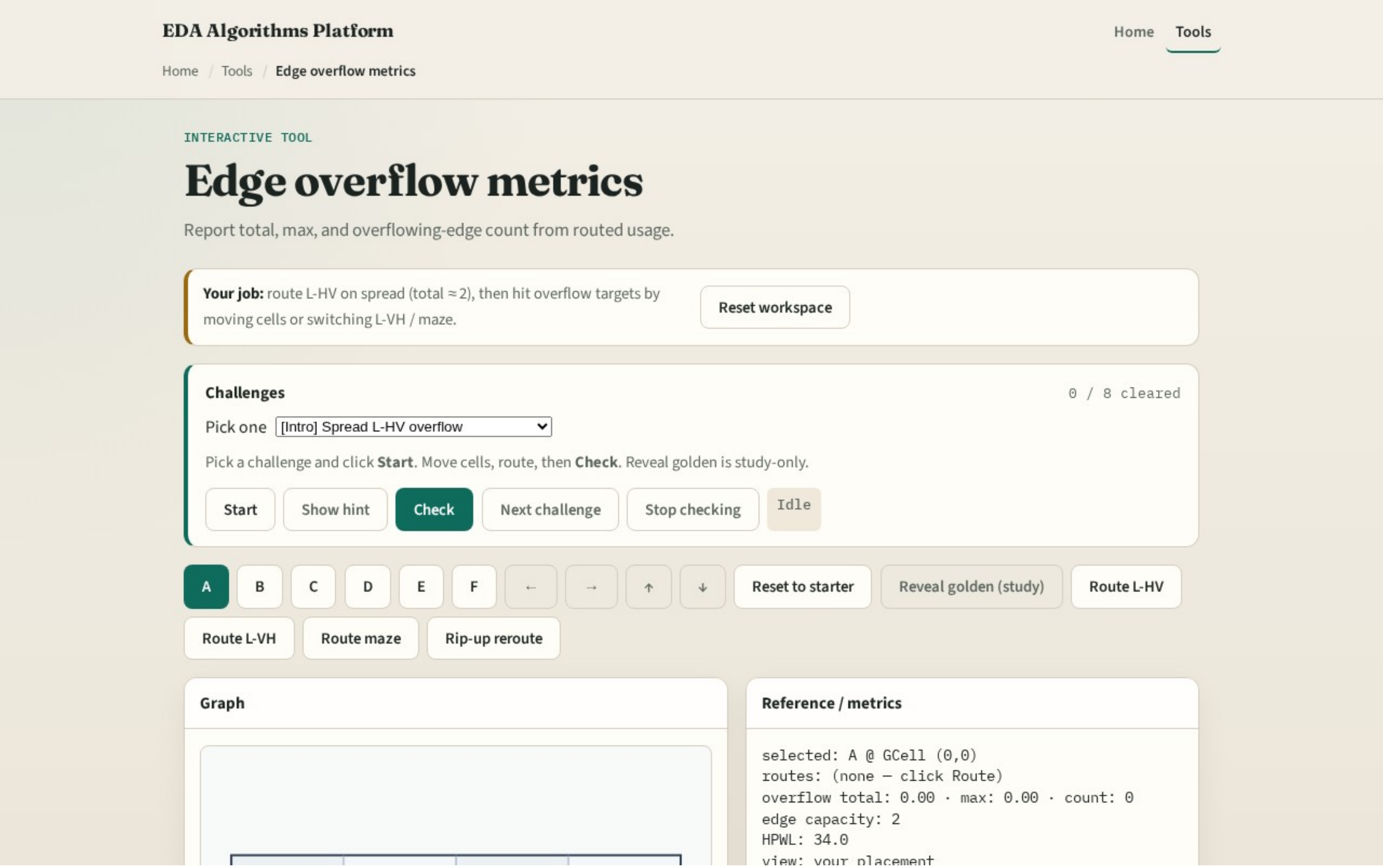
Move pins or switch HV/VH/maze to clear thresholds.

- Check scores learner
- No golden mode

```
Browser challenges
```

Hit targets

Browser lab track



Browser lab starter

Implement track

- Implement `edge_overflow(usage, capacity)`
- Call route_nets with mode l_hv on tiny_gr and assert total overflow is greater than zero

Pitfalls

- Computing overflow before summing all nets
- Using tile demand from congestion instead of edge usage
- Reporting negative overflow values

Your turn

- Hit overflow targets
- Next: rip-up the hottest net and maze reroute